SIMATS ENGINEERING

CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1709

Name	P. Ramesh
Register Number	192312443

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Code of Conduct:

I P. Ramesh (192312443) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *P. Ramesh*

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1)ANS: Decision Tree Classification

Objective

To implement and evaluate a Decision Tree classifier using the Iris dataset in Python. The experiment demonstrates data preprocessing, Gini Index-based decision tree construction, classification, and model evaluation.

A) Load, Pre-process and Split the Dataset

PROGRAM:

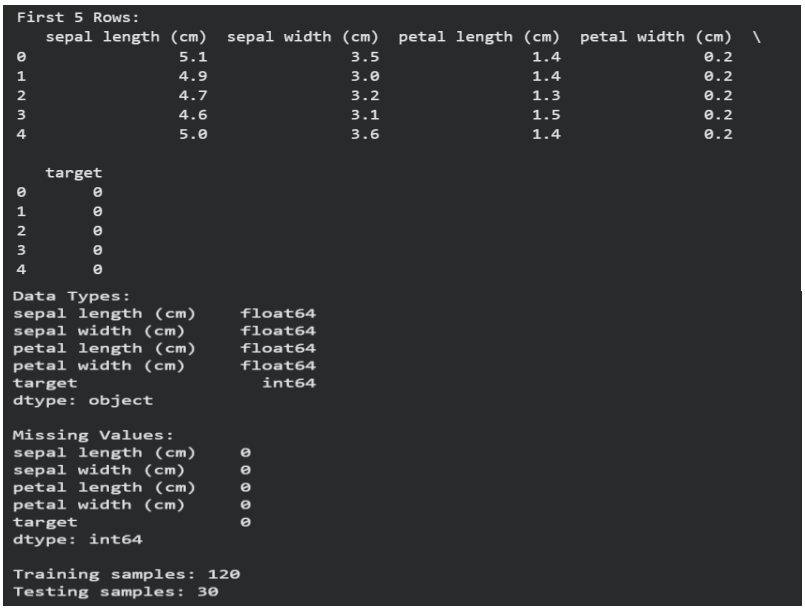
```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
# Load dataset
iris = load_iris()
# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
# Display first 5 rows
print("First 5 Rows:")
print(df.head())
# Display data types
print("\nData Types:")
print(df.dtypes)
# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())
# Handle missing values
df = df.fillna(df.mean(numeric_only=True))
# Separate features and target
X = df.drop("target", axis=1)
y = df["target"]
# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
```

)

print("\nTraining samples:", len(X_train))

print("Testing samples:", len(X_test))

OUTPUT:



```
First 5 Rows:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   target
0       0
1       0
2       0
3       0
4       0
Data Types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target                 int64
dtype: object

Missing Values:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
dtype: int64

Training samples: 120
Testing samples: 30
```

(b) Build Decision Tree Using Gini Index

PROGRAM:

from sklearn.tree import DecisionTreeClassifier, export_text

Create Decision Tree

model = DecisionTreeClassifier(

    criterion="gini",

    random_state=42

)

Train the model

model.fit(X_train, y_train)

Print tree structure

print("\nDecision Tree Structure:")

print(export_text(model, feature_names=list(X.columns)))

Identify root node

root_index = model.tree_.feature[0]

root_feature = X.columns[root_index]

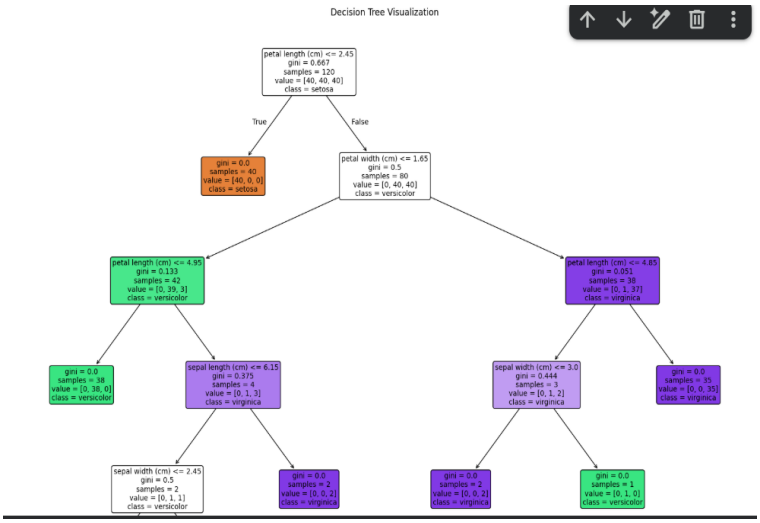
print("\nRoot Node:", root_feature)

print("Justification: The root node provides the best plit by minimizing Gini impurity.")

import matplotlib.pyplot as plt

```
from sklearn.tree import plot_tree
plt.figure(figsize=(20, 15))
plot_tree(model,
          feature_names=iris.feature_names,
          class_names=iris.target_names,
          filled=True,
          rounded=True,
          fontsize=10)
plt.title("Decision Tree Visualization")
plt.show()
```

OUTPUT:



(c) Evaluate the Model

PROGRAM:

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
# Predict test data
y_pred = model.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)
# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
# Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
# Find misclassified instances
print("\nMisclassified Instances:")
misclassified = 0
for i in range(len(y_test)):
    actual = y_test.iloc[i]
    predicted = y_pred[i]
    if actual != predicted:
        print(
            "Features:", X_test.iloc[i].values,
            "Actual:", iris.target_names[actual],
            "Predicted:", iris.target_names[predicted]
        )
        misclassified += 1
if misclassified == 0:
    print("No misclassified instances.")
```

OUTPUT:

```
Accuracy: 1.0000

Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        19
  versicolor       1.00      1.00      1.00        13
   virginica       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45


Misclassified Instances:
No misclassified instances.
```

2)ANS: Reinforcement Learning – Q-Learning

Objective

To implement Q-Learning for a simple 4×4 Grid World environment and observe the agent's learned policy.

(a) Environment and Q-Table

Environment

- Grid size: 4 × 4
- Total states: 16
- Actions:
  - U – Up
  - D – Down
  - L – Left
  - R – Right
- Goal: State 15
- Obstacle: State 5

Reward Structure

Situation	Reward
Reach Goal	+10
Normal step	−1
Hit obstacle	−5

PROGRAM:

```
import numpy as np
import random
import matplotlib.pyplot as plt
# Grid size
ROWS = 4
COLS = 4
STATES = ROWS * COLS
# Actions
actions = ['U', 'D', 'L', 'R']
# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 0.1
episodes = 100
```

```
# Goal and obstacle
goal = 15
obstacle = 5
# Q-table
Q = np.zeros((STATES, len(actions)))
# Convert state to row, column
def position(state):
    return divmod(state, COLS)
# Get next state and reward
def step(state, action):
    r, c = position(state)
    if action == 'U':
        r = max(0, r - 1)
    elif action == 'D':
        r = min(ROWS - 1, r + 1)
    elif action == 'L':
        c = max(0, c - 1)
    elif action == 'R':
        c = min(COLS - 1, c + 1)
    next_state = r * COLS + c
    if next_state == goal:
        reward = 10
    elif next_state == obstacle:
        reward = -5
    else:
        reward = -1
    return next_state, reward
# Choose action using epsilon-greedy
def choose_action(state):
    if random.random() < epsilon:
        return random.randint(0, 3)
    return np.argmax(Q[state])
# Store Q-values
saved_Q = {}
rewards = []
```

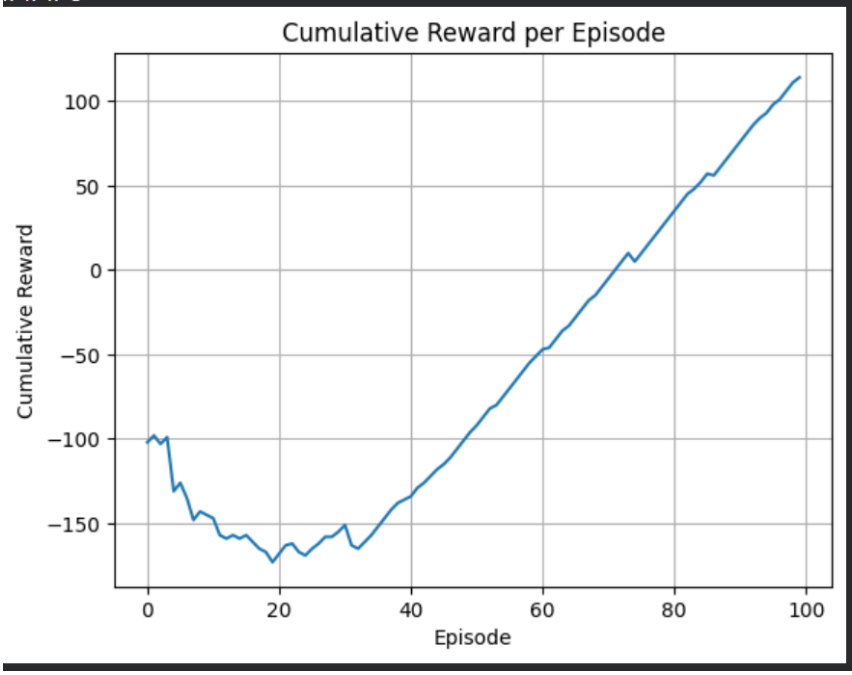
```
# Q-Learning
for episode in range(1, episodes + 1):
    state = 0
    total_reward = 0
    for step_count in range(100):
        action = choose_action(state)
        next_state, reward = step(
            state, actions[action]
        )
        # Q-Learning update
        Q[state, action] = Q[state, action] + alpha * (
            reward +
            gamma * np.max(Q[next_state]) -
            Q[state, action]
        )
        total_reward += reward
        state = next_state

        if state == goal:
            break
    rewards.append(total_reward)
    # Save Q-values at required episodes
    if episode in [1, 50, 100]:
        saved_Q[episode] = Q.copy()
# Display Q-values for two representative states
print("Q-Values for State 0 and State 10")
for episode in [1, 50, 100]:
    print("\nEpisode:", episode)
    print("State 0:", saved_Q[episode][0])
    print("State 10:", saved_Q[episode][10])
# Final Q-table
print("\nFinal Q-Table:")
print(np.round(Q, 2))
# Display learned policy
print("\nFinal Learned Policy:")
```

```
for state in range(STATES):
    if state == goal:
        print("G", end=" ")
    elif state == obstacle:
        print("X", end=" ")
    else:
        best_action = actions[np.argmax(Q[state])]
        print(best_action, end=" ")
    if (state + 1) % COLS == 0:
        print()
# Plot cumulative reward
cumulative_reward = np.cumsum(rewards)
plt.plot(cumulative_reward)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Cumulative Reward per Episode")
plt.grid()
plt.show()
```

OUTPUT:

```
Q-Values for State 0 and State 10

Episode: 1
State 0: [-0.199 -0.28  -0.199 -0.19 ]
State 10: [-0.1   -0.1   -0.109  0.   ]

Episode: 50
State 0: [-2.13543094 -2.14176004 -2.21797069 -2.13681107]
State 10: [-0.201785    7.19116287 -0.21105544  0.32257951]

Episode: 100
State 0: [-2.27391578 -2.1576146  -2.21797069  0.7148176 ]
State 10: [0.15852316 7.98722477 0.08146772 0.80789202]

Final Q-Table:
[[-2.27 -2.16 -2.22  0.71]
 [-1.38 -2.08 -1.57  2.5 ]
 [-0.82  4.31 -0.64 -0.93]
 [-0.39  0.15 -0.55 -0.39]
 [-1.49 -1.12 -1.56 -1.75]
 [-0.31 -0.21 -0.48  0.99]
 [-0.11  6.12 -1.66 -0.2 ]
 [-0.2   3.19 -0.11 -0.16]
 [-0.95 -0.87 -0.95  0.85]
 [-0.5  -0.3  -0.3   4.33]
 [ 0.16  7.99  0.08  0.81]
 [-0.1   8.15  0.    0.  ]
 [-0.54 -0.48 -0.49 -0.28]
 [-0.2  -0.29 -0.33  2.02]
 [ 0.96  2.72 -0.11 10.  ]
 [ 0.    0.    0.    0.  ]]
```

```
Final Learned Policy:
R R D D
D X D D
R R D D
R R R G
```



(b) Q-Table Evolution

Run the Q-Learning algorithm for 100 episodes and record the Q-values for State 0 and State 10 at Episodes 1, 50, and 100**.**

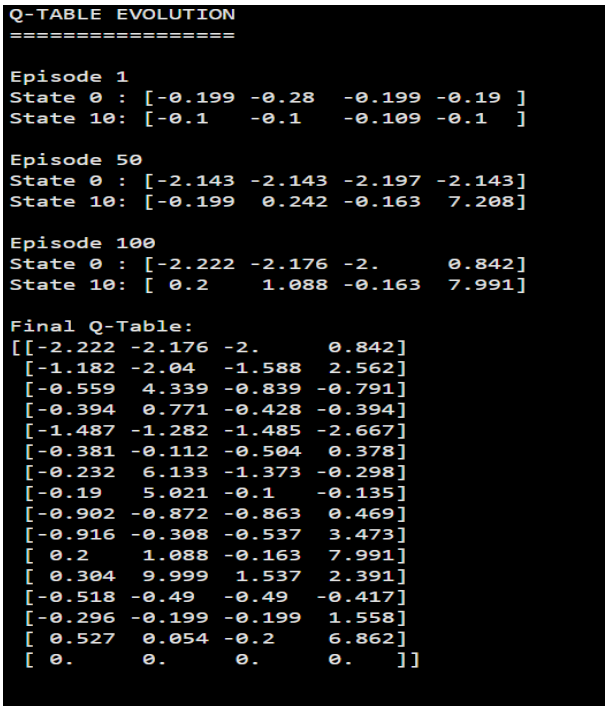
PROGRAM:

```
import numpy as np
import random

# Grid settings
ROWS = 4
COLS = 4
STATES = 16
# Actions: Up, Down, Left, Right
actions = ['U', 'D', 'L', 'R']
# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 0.1
episodes = 100
# Goal and obstacle
goal = 15
```

```
obstacle = 5
# Initialize Q-table
Q = np.zeros((STATES, 4))
# Get next state and reward
def step(state, action):
    r, c = divmod(state, COLS)
    if action == 0:       # Up
        r = max(0, r - 1)
    elif action == 1:     # Down
        r = min(ROWS - 1, r + 1)
    elif action == 2:     # Left
        c = max(0, c - 1)
    elif action == 3:     # Right
        c = min(COLS - 1, c + 1)
    next_state = r * COLS + c
    if next_state == goal:
        reward = 10
    elif next_state == obstacle:
        reward = -5
    else:
        reward = -1
    return next_state, reward
# Store Q-values
saved_values = {}
# Q-Learning
for episode in range(1, episodes + 1):
    state = 0
    for _ in range(100):
        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[state])
        next_state, reward = step(state, action)
        # Q-learning update
        Q[state, action] += alpha * (
```

```
            reward + gamma * np.max(Q[next_state])
            - Q[state, action]
        )
        state = next_state
        if state == goal:
          break
    # Save Q-values at Episodes 1, 50 and 100
    if episode in [1, 50, 100]:
        saved_values[episode] = Q.copy()
# Display Q-values
print("Q-TABLE EVOLUTION")
print("=================")
for episode in [1, 50, 100]:
    print("\nEpisode", episode)
    print("State 0 :", np.round(saved_values[episode][0], 3))
    print("State 10:", np.round(saved_values[episode][10], 3))
# Display final Q-table
print("\nFinal Q-Table:")
print(np.round(Q, 3))
```

OUTPUT:



```
Q-TABLE EVOLUTION
=================

Episode 1
State 0 : [-0.199 -0.28  -0.199 -0.19 ]
State 10: [-0.1   -0.1   -0.109 -0.1  ]

Episode 50
State 0 : [-2.143 -2.143 -2.197 -2.143]
State 10: [-0.199  0.242 -0.163  7.208]

Episode 100
State 0 : [-2.222 -2.176 -2.     0.842]
State 10: [ 0.2    1.088 -0.163  7.991]

Final Q-Table:
[[-2.222 -2.176 -2.     0.842]
 [-1.182 -2.04  -1.588  2.562]
 [-0.559  4.339 -0.839 -0.791]
 [-0.394  0.771 -0.428 -0.394]
 [-1.487 -1.282 -1.485 -2.667]
 [-0.381 -0.112 -0.504  0.378]
 [-0.232  6.133 -1.373 -0.298]
 [-0.19   5.021 -0.1   -0.135]
 [-0.902 -0.872 -0.863  0.469]
 [-0.916 -0.308 -0.537  3.473]
 [ 0.2    1.088 -0.163  7.991]
 [ 0.304  9.999  1.537  2.391]
 [-0.518 -0.49  -0.49  -0.417]
 [-0.296 -0.199 -0.199  1.558]
 [ 0.527  0.054 -0.2    6.862]
 [ 0.     0.     0.     0.   ]]
```

(c) Final Learned Policy and Cumulative Reward

Use the following code after running the Q-Learning algorithm

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
# Extract final learned policy
print("Final Learned Policy:")
print("---------------------")
symbols = ['U', 'D', 'L', 'R']
for state in range(16):
    if state == goal:
        action = 'G'
    elif state == obstacle:
        action = 'X'
    else:
        action = symbols[np.argmax(Q[state])]
    print(action, end=" ")
    if (state + 1) % 4 == 0:
        print()
# Calculate cumulative reward
cumulative_reward = np.cumsum(rewards)
# Plot cumulative reward
plt.figure(figsize=(8, 5))
plt.plot(cumulative_reward)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Cumulative Reward per Episode")
plt.grid(True)
plt.show()
```

OUTPUT:

```
Final Learned Policy:
----------------------
R R D D
D X D D
R R R D
R R R G
```